

Intro to Deep Learning

Lecture 11: Tokenization and Transformer Pretraining

Stanislav Don

DS212 - Intro to Deep Learning

Today

By the end of the lecture, you should be able to answer:

1. Why do pretrained language models exist?
2. What is subword tokenization?
3. What are token IDs and attention masks?
4. How is BERT pretrained?
5. How is GPT pretrained?
6. What is the difference between encoder and decoder models?

Lecture rhythm: text -> tokens/IDs -> BERT objective -> GPT objective -> HF workflow

Recap From Lecture 08

Transformers process sequences of tokens.

Attention works on embeddings:

```
X: [B, T, D]
```

But real text starts as strings:

```
"Paris is the capital of France."
```

Today:

```
text -> tokens -> IDs -> embeddings -> transformer
```

Part 1

Why pretraining?

Why Pretrained Language Models Exist

Text models need language knowledge:

- grammar
- common facts
- word meanings
- style patterns
- long-range dependencies
- task-specific formats

Training from scratch for every task is expensive.

So we pretrain once, then adapt or use the model later.

Pretraining

Pretraining means:

```
train a model on a broad general objective  
using a large amount of data
```

The model learns general representations.

Later we can use it for:

- inference
- fine-tuning
- prompting
- embeddings

Pretraining To Downstream Use

Pretraining learns general language ability before downstream use



Text Is Not Already Numbers

Neural networks need tensors.

Raw text must become integer IDs:

```
"deep learning"  
-> tokens  
-> token IDs  
-> embedding vectors
```

The tokenizer defines the mapping between strings and model inputs.

Model and tokenizer belong together.

Part 2

Tokenization and model inputs

Tokenization Choices

Possible token units:

characters
words
subwords
bytes

Whole words are intuitive but problematic:

- huge vocabulary
- unknown rare words
- hard handling of new words
- punctuation and spacing issues

Modern LMs usually use subword tokenization.

Subword Tokenization

Subword tokenization splits text into reusable pieces.

Example intuition:

```
"unbelievable"  
-> "un", "believ", "able"
```

Benefits:

- handles rare words
- keeps vocabulary manageable
- reuses common pieces
- avoids many unknown tokens

BPE Intuition

BPE-like tokenizers start with small units and merge frequent pairs.

Intuition:

```
common sequences become single tokens  
rare words remain split into pieces
```

We do not need to implement BPE in this course.

We need to understand why tokenizers may split words in surprising ways.

Subword Tokenization: Picture

Subword tokenization splits rare words into reusable pieces



Exact splits depend on the tokenizer checkpoint.

Tokenizer Types

Same text, different tokenizer behavior:

```
"unbelievable cats"
```

Tokenizer type	Illustrative output
Word tokenizer	<code>["unbelievable", "cats"]</code>
WordPiece	<code>["un", "##believable", "cat", "##s"]</code>
BPE / byte-level BPE	<code>["un", "believable", " cats"]</code>
SentencePiece / Unigram	<code>["_un", "believable", "_cats"]</code>
Character / byte	<code>["u", "n", "b", "..."]</code>

Exact tokens depend on the trained tokenizer vocabulary.

Token IDs

Tokens are converted to integer IDs.

Example:

```
tokens:    ["the", "cat", "sat"]  
input_ids: [1996, 4937, 2938]
```

The IDs index rows in the embedding table.

Then the model receives vectors, not strings.

Embedding Table

The embedding layer maps token IDs to vectors.

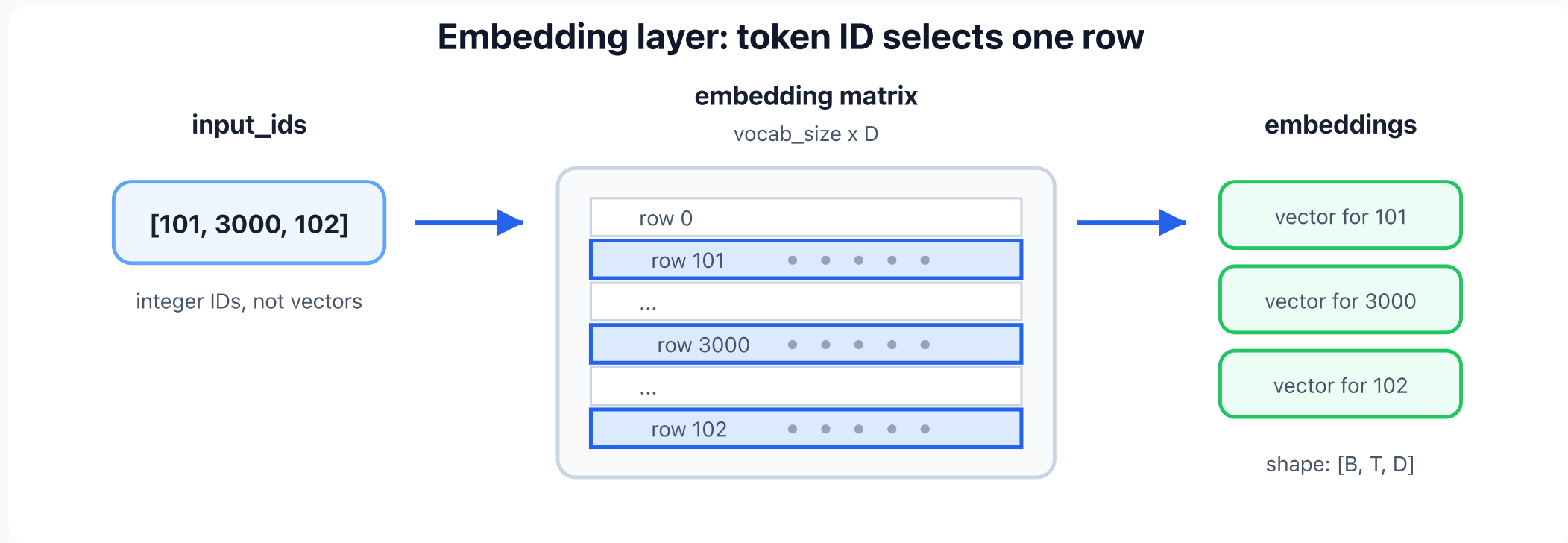
```
input_ids: [B, T]  
embeddings: [B, T, D]
```

Conceptually:

```
token ID -> lookup vector
```

During pretraining, these embeddings are learned with the model.

Embedding Lookup: Picture



The embedding table is a learned matrix.

Token IDs select rows from that matrix.

Special Tokens

Tokenizers often add special tokens.

BERT-style examples:

```
[CLS]  sequence-level representation  
[SEP]  separator  
[MASK] masked token for pretraining/inference  
[PAD]  padding
```

GPT-style tokenizers behave differently and often use an end-of-sequence token.

Attention Mask

The attention mask tells the model which tokens are real.

Example:

```
1 = real token  
0 = padding token
```

Why needed?

```
batches require equal length  
shorter sequences are padded  
model should ignore padding
```

This is not the same as causal masking.

Tokenizer Output

Tokenizer output contains tensors the model expects

tokens	input_ids	attention_mask	meaning
[CLS]	101	1	special token
Paris	3000	1	real token
[SEP]	102	1	separator
[PAD]	0	0	ignored padding

`tokenizer(...)` returns tensors such as `input_ids` and `attention_mask`.

These tensors can be passed to the model as keyword arguments.

Positional Information

Self-attention alone does not know token order.

The model needs positional information so that:

```
"dog bites man"
```

and:

```
"man bites dog"
```

are not treated as the same sequence.

Transformers add positional information to token embeddings.

Position Embeddings: Picture

Transformer input = token meaning + position



Without position information, self-attention has no built-in sense of word order.

The model input combines token identity and position.

This lets the transformer distinguish the same words in different orders.

Part 3

BERT-style masked language modeling

Two Pretraining Families

This lecture compares two important patterns:

BERT-style encoder models
GPT-style decoder models

BERT-style:

understand text with `bidirectional` context

GPT-style:

predict and generate text `left-to-right`

BERT-Style Pretraining

BERT-style models use masked language modeling.

Training task:

```
hide some tokens  
predict the missing tokens
```

Example:

```
Input:  Paris is the [MASK] of France.  
Target: capital
```

BERT can use context on both sides of the mask.

Bidirectional Context

For masked prediction, BERT can look at:

```
left context  
right context
```

Example:

```
Paris is the [MASK] of France.
```

The word after the mask helps.

This makes BERT natural for understanding and fill-in-the-blank tasks.

BERT Masked LM Output

For a masked position, the model predicts a distribution over the vocabulary.

Output:

```
score for every possible token
```

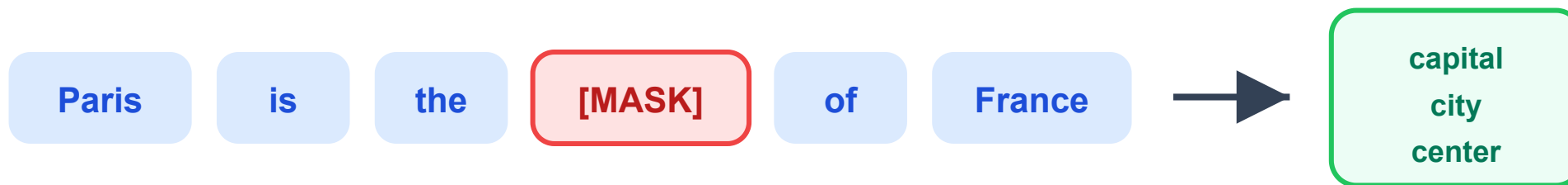
We choose top predictions:

```
capital  
city  
center  
...
```

Seminar 11 will inspect these top-k predictions.

Masked LM Objective

BERT-style pretraining predicts hidden tokens from both sides



Left and right context both help predict the masked position.

Part 4

GPT-style causal language modeling

GPT-Style Pretraining

GPT-style models use causal language modeling.

Training task:

```
predict the next token from previous tokens
```

Example:

```
Input:  The cat sat on the  
Target: mat
```

This matches generation naturally.

Causal Prediction

At position t , the model can use:

$$x_1, x_2, \dots, x_{t-1}$$

It should predict:

$$x_t$$

It must not look into the future.

This is why GPT-style models use causal masking.

GPT Loss

GPT-style pretraining minimizes next-token cross-entropy:

$$L_{GPT} = - \sum_t \log p(x_t \mid x_{<t})$$

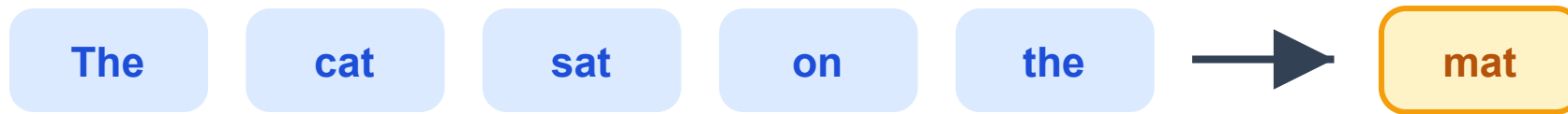
Meaning:

make the actual next token more probable
given all previous tokens

This objective is simple and very scalable.

Causal LM Objective

GPT-style pretraining predicts the next token



At each position, use previous tokens to predict the next one.

No future tokens are visible during training.

GPT-Style Generation

Generation repeats next-token prediction:

```
prompt -> predict next token  
append token -> predict next token  
append token -> ...
```

This is autoregressive generation.

In Seminar 11, DistilGPT2 will generate continuations from prompts.

The model is doing the same task it was pretrained for.

Temperature

Temperature changes sampling randomness.

Low temperature:

more predictable
less diverse
often more coherent

High temperature:

more diverse
more surprising
more risk of nonsense

Seminar 11 compares generation at different temperatures.

Part 5

Encoder/decoder behavior and workflow

Encoder Models

Encoder-style transformers process the full input at once.

They are good for:

- classification
- retrieval embeddings
- sentence understanding
- token classification
- masked prediction

Typical BERT-style behavior:

```
bidirectional context  
not naturally free-form generation
```

Decoder Models

Decoder-style transformers generate left-to-right.

They are good for:

- continuation
- chat-style generation
- summarization when prompted
- code generation
- instruction following after adaptation

Typical GPT-style behavior:

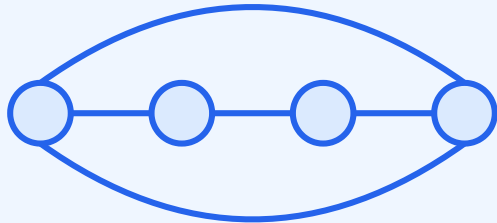
```
causal context  
natural autoregressive generation
```

Encoder vs Decoder: Picture

Encoder and decoder models use different attention patterns

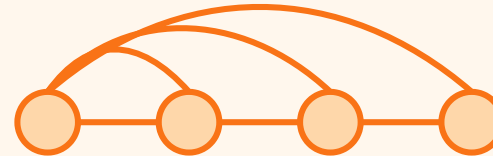
encoder / BERT-style

bidirectional attention



decoder / GPT-style

causal left-to-right attention



Pretraining vs Fine-Tuning vs Prompting

Pretraining:

learn general language ability from large data

Fine-tuning:

update model weights for a target task

Prompting:

give instructions/examples at inference time
without changing weights

These are different stages and usage modes.

Mini-Check: Continuation Setup

You want free-form continuation:

```
"In this lecture, we learned that"
```

You load:

```
AutoModelForMaskedLM.from_pretrained("distilbert-base-uncased")
```

What will feel awkward about this setup?

Which model family matches continuation better?

Answer

Masked language models are designed to fill masked positions, not freely continue text.

This setup expects inputs like:

```
Paris is the [MASK] of France.
```

For free-form continuation, use a GPT-style causal language model:

```
AutoModelForCausalLM.from_pretrained("distilgpt2")
```

It matches the left-to-right generation objective.

Hugging Face Mental Model

Most transformer workflows include:

```
tokenizer  
model  
inputs dictionary  
outputs object
```

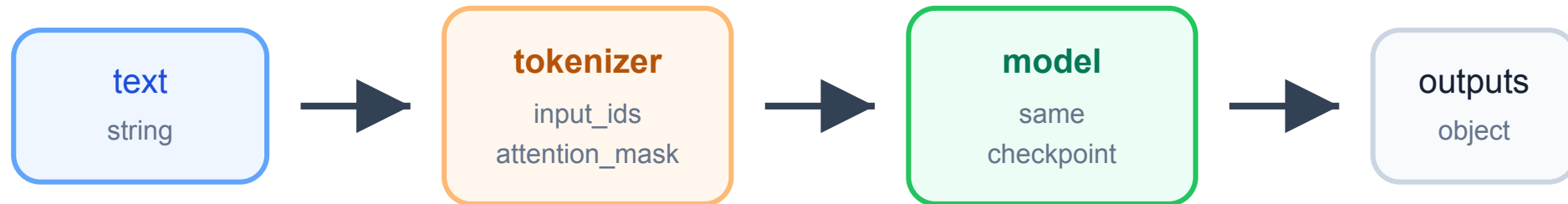
Typical flow:

```
inputs = tokenizer(text, return_tensors="pt")  
outputs = model(**inputs)
```

The tokenizer and model checkpoint must match.

Hugging Face Workflow

Tokenizer and model checkpoint must match



Do not mix tokenizer from one checkpoint with model weights from another.

Mini-Check: Strange Outputs

A model gives strange outputs after this code:

```
tokenizer = AutoTokenizer.from_pretrained("distilbert-base-uncased")  
model = AutoModelForCausalLM.from_pretrained("distilgpt2")
```

Shapes may still look valid.

What is the likely problem?

Answer

The tokenizer and model checkpoints do not match.

The tokenizer creates IDs using the DistilBERT vocabulary:

```
AutoTokenizer.from_pretrained("distilbert-base-uncased")
```

But the model interprets IDs using the DistilGPT2 vocabulary and weights:

```
AutoModelForCausalLM.from_pretrained("distilgpt2")
```

Use tokenizer and model from the same checkpoint family.

Auto Classes

Hugging Face `Auto*` classes choose the correct implementation from a checkpoint name.

Examples:

```
AutoTokenizer.from_pretrained(...)  
AutoModelForMaskedLM.from_pretrained(...)  
AutoModelForCausalLM.from_pretrained(...)
```

In Seminar 11:

```
DistilBERT -> masked LM  
DistilGPT2 -> causal LM
```

Distilled Models

DistilBERT and DistilGPT2 are smaller versions of larger models.

Why use them in class?

- faster inference
- lower memory use
- smoother Colab experience
- same core ideas as the larger family

They are good teaching models.

Do not confuse smaller with conceptually different.

Common Mistakes

Mistake 1:

`using a tokenizer from the wrong checkpoint`

Mistake 2:

`confusing attention mask with causal mask`

Mistake 3:

`expecting BERT to freely generate text`

Mistake 4:

`forgetting special tokens and padding behavior`

Debugging Checklist

Before running a transformer model, check:

- tokenizer checkpoint matches model checkpoint
- `input_ids` shape is `[B, T]`
- `attention_mask` shape is `[B, T]`
- special tokens make sense for the model family
- masked LM input contains exactly the intended `[MASK]`
- GPT-style generation has a pad/eos token set if needed

Seminar Bridge

In Seminar 11, you will:

1. load DistilBERT and DistilGPT2
2. compare subword tokenization
3. inspect `input_ids`, `attention_mask`, and special tokens
4. run masked prediction with DistilBERT
5. run generation with DistilGPT2
6. compare generation temperatures
7. summarize encoder vs decoder behavior

Main goal:

connect tokenization and pretraining objectives to model behavior

Takeaways

Pretrained language models learn from general text objectives.

Key ideas:

- tokenizers convert strings into model IDs
- subwords handle rare and new words
- GPT-style models learn next-token prediction
- BERT-style models learn masked-token prediction
- encoders and decoders are useful for different workflows

Next lecture:

`fine-tuning transformers for classification`